

pricing tech specs - final

MetaOrcha — Pricing & Payments Technical Specification

ID: EM-21 | Version: 4.1 | Status: Approved for MVP (Phase 1)

Prepared by: Azan Khan & Farzan Saqib | Confidential

Table of Contents

- [1. Platform Identity & Responsibility Model](#)
 - [2. Three-Phase Ecosystem Evolution](#)
 - [3. Phase 1 — MVP Pricing Primitives](#)
 - [4. Pricing Formulae](#)
 - [5. Routing Score & Algorithm](#)
 - [6. Credit & Settlement Model](#)
 - [7. Payment State Machine](#)
 - [8. Code Architecture — Monorepo Layout](#)
 - [9. common/pricing — Pure Library Spec](#)
 - [10. Gateway — Operational Payment Layer](#)
 - [11. SuperAgent — PaymentGuard Middleware](#)
 - [12. PnD — Routing Integration](#)
 - [13. Database Schema Changes](#)
 - [14. Registry Changelog — x402 Removal](#)
 - [15. Wallet Architecture \(Privy\)](#)
 - [16. Wallet UI](#)
 - [17. Phase 2 & Phase 3 — Forward Reference](#)
 - [18. Open Items](#)
-

1. Platform Identity & Responsibility Model

Identity

MetaOrcha is an **Intelligent Brokerage**. Users describe goals in natural language; the platform decomposes tasks, discovers best-fit agents from an open registry, orchestrates multi-agent workflows at runtime, and settles payments transparently. Three alternative identities were evaluated and rejected:

Identity	Analogy	Why Rejected
Distribution Channel	App Store	MetaOrcha selects agents at runtime — the user never browses
Infrastructure Provider	AWS / GCP	MetaOrcha sells execution, not compute hours
Passive Marketplace	eBay / Airbnb	MetaOrcha actively selects — it is not neutral

The platform adopts a **Partner + Contractor hybrid** with developers: developers have quality agency (their reliability determines ranking and volume) but not demand agency (the platform dispatches them at runtime, not the user).

Responsibility Model

MetaOrcha **guarantees execution**, not outcome. This is identical to the model used by every major AI service (ChatGPT, Copilot, Claude, OpenRouter). The industry has trained users to accept that AI outputs are probabilistic, not deterministic.

System-Level Guarantees (MetaOrcha's responsibility):

- The selected agent was healthy at dispatch time
- Planning & Discovery selected the best-fit agent based on available signals
- User credentials were securely injected via the VaultService
- The execution pipeline completed without infrastructure failure
- Payment was only charged when the agent returned a non-error response
- All telemetry (latency, success_rate, uptime) is transparently observable

User-Owned Outcomes (not MetaOrcha's guarantee):

- Whether the agent's output matches the user's subjective expectation
- Whether the user's prompt was sufficiently clear
- Whether third-party APIs accessed by the agent returned correct data
- Whether generated content is factually accurate (LLM hallucination risk)

Design consequence: Phase 1 triggers payment on system-level success (non-error response), not on output quality evaluation. The Evaluator Agent is explicitly **not part of Phase 1**. Output quality assessment is deferred to Phase 2's implicit feedback loop.

2. Three-Phase Ecosystem Evolution

Each phase builds on the data infrastructure of the previous. No phase is skippable.

Phase	Name	Core Capability	Pricing Basis	Ranking Signals
Phase 1 (v1)	System-Level Observability	Reliable execution, transparent billing	base_fee + token_cost	success_rate , p95_latency , uptime
Phase 2 (v1.5)	Implicit Feedback Loop	Behavioural learning from user sessions	base_fee + F:C ratio ranking	User retry, session abandonment, satisfaction signal, task completion rate
Phase 3 (v2)	Trained SuperAgent	Predictive orchestration, learned ranking model	Full stack + outcome_bonus	Fine-tuned selection model + GNN workflow optimiser

The Data Flywheel

Phase 1 collects raw invocation telemetry → Phase 2 mines behavioural signals from that data → Phase 3 trains a ranking model on Phase 2's quality signals. This compounding loop is the platform's core competitive moat and is extremely difficult to replicate without comparable traffic volume.

3. Phase 1 — MVP Pricing Primitives

3.1 Active Primitives

Primitive	Type	Set By	Applies To	Trigger
base_fee	Billing	Developer (manifest)	A2A / ACP agents only	Non-error response received
token_cost	Billing	Platform (fixed rate)	All agent types	SuperAgent output tokens per turn
success_rate	Ranking signal	Platform	All agents	Rolling 7-day: non-error / total invocations
p95_latency_ms	Ranking signal	Platform	All agents	Wall-clock per invocation, middleware timed
uptime	Ranking signal	Platform	All agents	Registry health check pass rate, 7-day rolling

MCP tool-servers: not charged. No pricing primitives apply. They are infrastructure primitives.

x402 protocol: permanently dropped. x402 requires upfront payment before execution. MetaOrcha charges after confirmed non-error response. The two models are incompatible. All x402 references are removed from registry, manifest schema, and EmergeSDK.

3.2 Deferred Primitives (Not Phase 1)

Primitive	Deferred To	Reason
fulfillment_score	Phase 2	Requires behavioural data to be meaningful
trust_score	Phase 2	Derived from dispute signals, insufficient volume at launch
outcome_bonus	Phase 3	Requires calibrated evaluation model trained on Phase 2 data
payload_billing	Phase 2+	Low ROI at current scale; adds DX friction
action_receipt	Phase 3	Requires EmergeSDK egress wrapper (not yet built)
Evaluator Agent	Phase 3	Premature — no training data to calibrate quality thresholds

3.3 Primitive Evaluation: The Three Criteria

Primitive	Measurable?	Right Incentive?	Can Be Rigged?
base_fee	✅ Declared in manifest; verified by non-error response	✅ Encourages competitive access pricing	Low — capped by market bounds
token_cost	✅ SuperAgent internal output-token counter	✅ Aligns MetaOrcha's LLM spend with revenue recovery	Very Low — platform counts, not agent-reported
success_rate	✅ Non-error responses / total invocations	✅ Penalises unreliable agents in ranking	Low — platform measured
p95_latency_ms	✅ Wall-clock per invocation	✅ Pressures developers to optimise response time	Very Low — platform measured
uptime	✅ Health check pass rate	✅ Excludes flaky agents from selection	Low — health checks are platform-initiated
fulfillment_score (deferred)	⚠️ Requires calibration	✅ Would penalise low-quality agents	Low once trained; high before training data exists
outcome_bonus (deferred)	⚠️ Requires verified fulfillment mechanism	✅ Rewards high-value agents	Medium — threshold gaming possible without Phase 2 data

Primitive	Measurable?	Right Incentive?	Can Be Rigged?
Capability-level pricing	✗ Rejected	✗ Phantom capability attack	High — trivially exploitable
Latency as billing primitive	✗ Rejected	✗ Penalises legitimate long-running tasks	N/A

4. Pricing Formulae

4.1 Per Agent Invocation (A2A / ACP)

```
execution_success = (status == "completed") AND (response is non-error, non-empty)
```

```
charge = base_fee      if execution_success == True
charge = $0.00         if execution_success == False (error / timeout / empty)
```

No evaluation. No schema check. System-level success signal only.

4.2 Platform Token Cost (SuperAgent output tokens only)

Covers MetaOrcha's own LLM inference across the workflow: ReAct reasoning turns, PnD queries, decomposition. **Input tokens (system prompts, templates) are never forwarded to the user.**

```
platform_token_cost = superagent_output_tokens × PLATFORM_TOKEN_RATE
```

PLATFORM_TOKEN_RATE is defined in `common/pricing/constants.py` (e.g. `$0.000002 / token`).

4.3 Turn Cost (what is deducted per ReAct turn)

```
turn_cost = Σ(base_fee_i for each successful invocation this turn)
            + platform_token_cost_this_turn
```

4.4 Revenue Split

```
developer_payout = base_fee × (1 - PLATFORM_SPREAD)      # e.g. 80%
platform_revenue = base_fee × PLATFORM_SPREAD            # e.g. 20%
                  + platform_token_cost                  # 100% platform
(cost recovery)
```

PLATFORM_SPREAD is defined in `common/pricing/constants.py`.

4.5 Market Bounds (user protection, Phase 1)

```
floor    = category_median × BASE_FEE_FLOOR_MULTIPLIER    # e.g. 0.10
ceiling  = category_median × BASE_FEE_CEILING_MULTIPLIER   # e.g. 5.00
```

- Agents **below floor**: auto-excluded from registry (scam signal)
- Agents **above ceiling**: excluded from auto-routing; require explicit user selection

Category medians are computed nightly by the Gateway settlement cron once sufficient data exists. Until then, bounds checks are skipped (bootstrap mode).

5. Routing Score & Algorithm

5.1 Phase 1 Routing Score

Phase 1 uses only system-level signals. No fulfillment score, no trust score.

```
# Bootstrap (< BOOTSTRAP_EXECUTION_COUNT invocations):
Routing_Score = success_rate / (p95_latency_ms × base_fee_usd)

# Defaults for new agents (from constants):
#   BOOTSTRAP_SUCCESS_RATE    = 0.70
#   BOOTSTRAP_LATENCY_MS      = 5000
#   norm_latency               = 1.0  (until registry medians computed)
#   norm_base_fee              = 1.0  (until registry medians computed)

# Post-bootstrap:
Routing_Score = (success_rate × uptime_score)
               / (norm_latency × norm_base_fee)

norm_latency  = agent.p95_latency_ms / registry_median_latency_ms
norm_base_fee = agent.base_fee_usd   / registry_median_base_fee_usd
```

Phase 2 upgrade: `success_rate` is replaced with the F:C (Fulfillment:Cost) ratio derived from implicit behavioural signals. The formula structure stays identical; only the quality signal changes.

5.2 PnD Pipeline — Routing Integration

Routing score is injected **after cross-encoder reranking, before the Semantic Analysis LLM call**. The Semantic LLM system prompt is updated to honour routing scores in its final selection decision.

Step	Stage	Detail
1	Hard filter (SQL)	<code>health_status = HEALTHY</code> , <code>base_fee</code> within bounds, <code>uptime > UPTIME_FLOOR</code>
2	Hybrid retrieval	BM25 + dense vector search, RRF fusion → top-50 candidates
3	Cross-encoder reranking	Produces ranked top-5 by semantic match score
4	Routing score injection	<code>Routing_Score</code> computed for each of top-5, passed to Semantic LLM
5	Semantic LLM selection	Haiku selects best agent; system prompt: <i>"honour routing score — prefer agents with higher scores when semantic match is comparable"</i>
6	Tie-break	Lower <code>base_fee</code> wins
7	Retry fallback	If selected agent hits <code>MAX_RETRIES</code> : re-score remaining pool, excluding failed agent

5.3 Retry Policy

`MAX_RETRIES = 3` (defined in constants)

Per retry:

- \$0 charged (no `base_fee`, tokens not billed on failed invocation)
- Retry counted against agent's `success_rate`

After `MAX_RETRIES`:

- Agent dropped from this workflow execution
- `success_rate` updated (negative signal)
- PnD selects next-best candidate from remaining pool

6. Credit & Settlement Model

6.1 Credit System

Users hold an internal credit balance denominated in USD. Credits are funded via:

- **Stripe** (credit card, PayPal) — primary user-facing method
- **USDC deposit** to Privy custodial wallet — crypto-native option

The user **never interacts with a wallet, blockchain, or crypto interface directly** for normal usage. Credits are the only unit of account in the product UI.

6.2 Per-Turn Credit Check (ReAct Loop Compatible)

Because the ReAct loop is dynamic, cost cannot be estimated upfront. Credits are checked and deducted after each turn.

```
# Before each turn:
if user.arrears_flag == True OR user.credits_balance <= 0:
    pause → HITL top-up prompt
    # workflow resumes only after top-up clears arrears

# After each turn (in PaymentSettlement middleware, step 6.5):
turn_cost = Σ(base_fee_i for successful invocations this turn)
            + platform_token_cost_this_turn

if credits_balance >= turn_cost:
    credits_balance -= turn_cost
else:
    shortfall = turn_cost - credits_balance
    credits_balance = 0
    arrears_balance += shortfall
    arrears_flag = True # system wallet absorbs shortfall
```

6.3 Arrears Recovery on Top-Up

```
if arrears_flag == True:
    min_topup = arrears_balance × (1 + ARREARS_INTEREST_RATE) # e.g. 1.05
    if top_up_amount < min_topup:
        reject → "Minimum top-up required: ${min_topup}"
    else:
        credits_balance = top_up_amount - min_topup
        arrears_balance = 0
        arrears_flag = False
```

6.4 Daily Settlement (Cron — runs in Gateway)

```
# Runs at SETTLEMENT_CRON_SCHEDULE (e.g. '0 0 * * *')
# Protected by Redis distributed lock — only one Gateway instance executes

developer_payout_i = Σ(base_fee_j × (1 - PLATFORM_SPREAD)) # per
developer
→ batch USDC transfer to developer custodial wallets (Avalanche Fuji
Testnet)
→ Transaction records: PENDING → SETTLED
→ Pending_settlement amount released from user accounts
```

7. Payment State Machine

EXECUTING → invocation dispatched, latency timer started
PaymentGuard reserves base_fee from credits (soft hold in Redis)

```
|— timeout / error / empty → FAILED
|   credits hold released
|   base_fee = $0
|   success_rate penalty applied
|   retry (up to MAX_RETRIES) or agent dropped
|
|— non-error response received → SETTLING
   credits deducted (turn_cost)
   Transaction record written: status = PENDING
   developer_payout queued
   agent success_rate, p95_latency updated
```

PENDING → awaiting daily settlement cron

SETTLED → developer payout executed on-chain
Transaction status = SETTLED

DISPUTED (future, Phase 2+)
→ user flags outcome
does NOT trigger refund at Phase 1
updates agent quality signals for Phase 2 feedback loop

8. Code Architecture — Monorepo Layout

The Problem

Pricing logic spans multiple services. Without a deliberate structure:

- Formula changes require edits across `superagent/` , `gateway/` , `planning-discovery/` , `registry/`
- Constants diverge between services
- Operational code (DB writes, crons) gets tangled with pure calculation code

The Split: Two Distinct Layers

Layer 1 — Pure/Functional logic (`common/pricing`)

Stateless. Given inputs, returns outputs. No DB, no HTTP, no scheduler. Importable by any service with zero side effects.

Layer 2 — Operational logic (service-owned)

Stateful. Touches DB, emits events, runs scheduled jobs. Owned by the service that is the right

deployment boundary for that operation.

Monorepo Layout

```
common/
├─ db/
├─ kafka/
├─ llm/
└─ pricing/
    ├─ __init__.py
    └─ constants.py
of truth
    └─ formulae.py
split_revenue(), turn_cost()
    └─ models.py
CostBreakdown, RoutingCandidate
    └─ bounds.py
market_ceiling(), uptime_gate()

services/
├─ gateway/
│   └─ wallet/
withdraw routes
│   └─ jobs/
│       └─ settlement.py
distributed lock
│   └─ metrics_refresh.py
computation
│
├─ superagent/
│   └─ pricing/
│       └─ guard.py
– credit hold
│   └─ settlement.py
6.5) – deduct + write TX
│
├─ planning-discovery/
│   └─ ranking.py
common/pricing
│
└─ registry/
    └─ validation.py
for manifest validation
```

← NEW common package

← ALL magic numbers – single source

← Pure functions: `routing_score()`,

← Dataclasses: `InvocationRecord`,

← Pure: `market_floor()`,

← Privy client, balance, fund,

← APScheduler daily cron + Redis

← Nightly registry median

← PaymentGuard middleware (step 2.5)

← PaymentSettlement middleware (step 6.5)

← Imports `routing_score()` from

← Imports bounds from `common/pricing`

Key Architectural Principles

common/pricing has zero operational dependencies. No DB client, no Redis, no HTTP. If `AsyncSession` is imported into `formulae.py`, the architecture is broken. The package is a pure

calculation library — testable with `pytest` in complete isolation.

Each service owns its own operational glue. SuperAgent owns the middleware that writes to DB after an invocation. Gateway owns the settlement cron. Both call into `common/pricing` for the numbers; they own the side effects.

Constants is the single source of truth for policy. Every magic number — `PLATFORM_SPREAD`, `MAX_RETRIES`, `BOOTSTRAP_SUCCESS_RATE` — lives only in `common/pricing/constants.py`. Services never define their own copy. Updating pricing policy = one file, one PR.

Settlement cron runs in Gateway with a Redis distributed lock. Gateway may have multiple instances. The lock ensures only one executes settlement at a time. This pattern mirrors the existing PnD leader election implementation.

```
# gateway/jobs/settlement.py
async def run_settlement():
    lock = await redis.set(
        "settlement:lock",
        value=gateway_instance_id,
        nx=True,          # only set if not exists
        ex=300,          # 5-minute TTL
    )
    if not lock:
        return           # another instance already running

    try:
        await process_pending_transactions()
        await execute_developer_payouts()
    finally:
        await redis.delete("settlement:lock")
```

9. common/pricing — Pure Library Spec

constants.py

```
import os
from decimal import Decimal

# — Kill switch
PRICING_ENABLED = os.getenv("PRICING_ENABLED", "true").lower()
== "true"

# — Platform economics
```

```

PLATFORM_SPREAD          = Decimal("0.20")          # 20% brokerage
commission
PLATFORM_TOKEN_RATE      = Decimal("0.000002")      # $/output token
ARREARS_INTEREST_RATE    = Decimal("0.05")          # 5% on outstanding
debt

# — Execution policy


---


MAX_RETRIES              = 3
UPTIME_FLOOR             = 0.80                    # Hard exclusion
below 80% uptime
SUCCESS_RATE_FLOOR       = 0.50                    # Hard exclusion
below 50% success rate

# — Routing bootstrap defaults


---


BOOTSTRAP_SUCCESS_RATE   = 0.70
BOOTSTRAP_LATENCY_MS     = 5000
BOOTSTRAP_EXECUTION_COUNT = 50                    # Threshold to exit
bootstrap phase

# — Market bounds


---


BASE_FEE_FLOOR_MULTIPLIER = Decimal("0.10")
BASE_FEE_CEILING_MULTIPLIER = Decimal("5.00")

# — Settlement schedule


---


SETTLEMENT_CRON_SCHEDULE = "0 0 * * *"            # daily midnight UTC
METRICS_REFRESH_SCHEDULE = "0 1 * * *"            # nightly at 01:00
UTC

```

formulae.py

```

from decimal import Decimal
from .constants import PLATFORM_SPREAD, PLATFORM_TOKEN_RATE

def routing_score(
    success_rate: float,
    uptime_score: float,
    p95_latency_ms: int,
    base_fee_usd: Decimal,
    norm_latency: float = 1.0,
    norm_base_fee: float = 1.0,
) -> float:
    """
    Phase 1 routing score. Pure function – no side effects.
    norm_latency and norm_base_fee default to 1.0 during bootstrap.
    """

```

```

"""
quality = success_rate * uptime_score
cost     = (p95_latency_ms / 1000) * float(base_fee_usd)
denominator = (norm_latency * norm_base_fee * cost)
if denominator == 0:
    return 0.0
return quality / denominator

def split_revenue(base_fee: Decimal) -> tuple[Decimal, Decimal]:
    """Returns (developer_payout, platform_cut)."""
    platform_cut = (base_fee *
PLATFORM_SPREAD).quantize(Decimal("0.000001"))
    developer_payout = base_fee - platform_cut
    return developer_payout, platform_cut

def compute_turn_cost(
    base_fees: list[Decimal],
    platform_output_tokens: int,
) -> Decimal:
    """Total cost charged to user for one ReAct turn."""
    token_cost = Decimal(str(platform_output_tokens)) *
PLATFORM_TOKEN_RATE
    return sum(base_fees, Decimal("0")) + token_cost

def compute_arrears_minimum_topup(arrears: Decimal, interest_rate:
Decimal) -> Decimal:
    """Minimum top-up required to clear arrears including interest."""
    return (arrears * (Decimal("1") +
interest_rate)).quantize(Decimal("0.01"))

```

models.py

```

from dataclasses import dataclass, field
from decimal import Decimal
from datetime import datetime

@dataclass
class InvocationRecord:
    call_id: str
    session_id: str
    agent_id: str
    user_id: str
    base_fee: Decimal
    latency_ms: int
    execution_success: bool

```

timestamp: datetime

```
@dataclass
class CostBreakdown:
    base_fees_total:       Decimal
    platform_token_cost:   Decimal
    turn_cost:            Decimal
    developer_payout:     Decimal
    platform_cut:          Decimal
```

```
@dataclass
class RoutingCandidate:
    agent_id:             str
    base_fee_usd:         Decimal
    success_rate:         float
    uptime_score:          float
    p95_latency_ms:       int
    execution_count:       int
    routing_score:         float = 0.0
```

bounds.py

```
from decimal import Decimal
from .constants import BASE_FEE_FLOOR_MULTIPLIER,
BASE_FEE_CEILING_MULTIPLIER

def market_floor(category_median: Decimal) -> Decimal:
    return category_median * BASE_FEE_FLOOR_MULTIPLIER

def market_ceiling(category_median: Decimal) -> Decimal:
    return category_median * BASE_FEE_CEILING_MULTIPLIER

def is_within_bounds(base_fee: Decimal, category_median: Decimal) ->
tuple[bool, str]:
    """Returns (within_bounds, reason). category_median=None skips check
    (bootstrap)."""
    if category_median is None:
        return True, "bootstrap"
    if base_fee < market_floor(category_median):
        return False, "below_floor"
    if base_fee > market_ceiling(category_median):
        return False, "above_ceiling"
    return True, "ok"
```

```
def uptime_gate(uptime_score: float, floor: float) -> bool:
    return uptime_score >= floor
```

10. Gateway — Operational Payment Layer

Module Structure

```
services/gateway/
├─ wallet/
│   └─ privy_client.py          # Wrapper: create_wallet, get_balance,
transfer
│   └─ routes.py                # GET /wallet/balance, POST /wallet/fund,
POST /wallet/withdraw
│   └─ webhook.py               # Privy deposit webhook → credit
user.credits_usd in DB
└─ jobs/
    └─ settlement.py            # Daily settlement cron + Redis lock
    └─ metrics_refresh.py       # Nightly: compute registry medians,
update agent metrics
```

settlement.py responsibilities

- Acquire Redis distributed lock before execution
- Query all Transaction records with status = PENDING
- Aggregate developer_payout per agent.user_id (developer)
- Execute USDC transfers via privy_client.transfer()
- Update Transaction.status = SETTLED, set settled_at
- Release Redis lock
- Log settlement summary

metrics_refresh.py responsibilities

- Compute p95_latency_ms per agent from agent_invocations (rolling 7-day)
 - Compute success_rate per agent from agent_invocations (rolling 7-day)
 - Compute uptime_score from registry health check logs (rolling 7-day)
 - Compute registry-wide median_latency_ms and median_base_fee_usd per task category
 - Write results back to agents table and a registry_stats table
-

11. SuperAgent — PaymentGuard Middleware

Two new middleware steps are inserted into the existing 7-step `ExecutionMiddleware` pipeline:

1. ROUTE & RESOLVE
2. INPUT GUARD
- 2.5 — PAYMENT GUARD (NEW)

```
    if not PRICING_ENABLED: skip
    Check user.credits_balance > 0 and not user.arrears_flag
    If insufficient: raise NodeInterrupt → HITL top-up prompt
    Soft-reserve base_fee in Redis (key: reserve:{session_id}:{call_id})
```

3. PRE-FLIGHT CHECKS
4. HANDLER DISPATCH
5. OUTPUT NORMALIZER
6. CHECKLIST AUTO-UPDATE
- 6.5 — PAYMENT SETTLEMENT (NEW)

```
    if not PRICING_ENABLED: skip
    On execution_success:
        release Redis reserve
        emit pricing event → handled async by pricing/settlement.py
        (DB write: deduct credits, write Transaction(PENDING), update
 invocation record)
    On execution_failure:
        release Redis reserve (no charge)
        update agent success_rate (penalty)
7. (end)
```

pricing/guard.py

```
# services/superagent/pricing/guard.py
from common.pricing.constants import PRICING_ENABLED
from common.pricing.models import RoutingCandidate

async def payment_guard(
    user_id: str,
    agent: RoutingCandidate,
    session_id: str,
    call_id: str,
    db: AsyncSession,
    redis: Redis,
) -> None:
    if not PRICING_ENABLED:
        return

    user = await db.get(User, user_id)

    if user.arrears_flag:
        raise PaymentInterrupt("arrears", user.arrears_usd)
```

```

if user.credits_usd <= 0:
    raise PaymentInterrupt("insufficient_credits", Decimal("0"))

# Soft reserve in Redis (TTL = max agent timeout)
await redis.setex(
    f"reserve:{session_id}:{call_id}",
    value=str(agent.base_fee_usd),
    time=300,
)

```

pricing/settlement.py (SuperAgent side)

```

# services/superagent/pricing/settlement.py
from common.pricing.formulae import split_revenue, compute_turn_cost
from common.pricing.constants import PRICING_ENABLED

async def settle_invocation(
    invocation: InvocationRecord,
    platform_tokens: int,
    db: AsyncSession,
    redis: Redis,
) -> None:
    if not PRICING_ENABLED:
        return

    # Release reserve
    await redis.delete(f"reserve:{invocation.session_id}:{invocation.call_id}")

    if not invocation.execution_success:
        await update_agent_success_rate(invocation.agent_id,
        success=False, db=db)
        return

    developer_payout, platform_cut = split_revenue(invocation.base_fee)

    # Deduct credits
    await db.execute(
        update(User)
        .where(User.id == invocation.user_id)
        .values(credits_usd=User.credits_usd - invocation.base_fee)
    )

    # Write transaction record
    tx = Transaction(
        session_id      = invocation.session_id,
        user_id          = invocation.user_id,
        agent_id         = invocation.agent_id,
        call_id          = invocation.call_id,
    )

```

```

        base_fee          = invocation.base_fee,
        platform_cut      = platform_cut,
        developer_payout  = developer_payout,
        latency_ms        = invocation.latency_ms,
        status            = TxStatus.PENDING,
    )
    db.add(tx)

    # Update agent metrics
    await update_agent_metrics(invocation, db=db)
    await db.commit()

```

12. PnD — Routing Integration

```

# planning-discovery/ranking.py
from common.pricing.formulae import routing_score
from common.pricing.constants import (
    BOOTSTRAP_SUCCESS_RATE, BOOTSTRAP_LATENCY_MS,
    BOOTSTRAP_EXECUTION_COUNT,
    SUCCESS_RATE_FLOOR, UPTIME_FLOOR,
)
from common.pricing.bounds import uptime_gate
from common.pricing.models import RoutingCandidate

def score_candidates(
    candidates: list[RoutingCandidate],
    registry_median_latency_ms: float | None,
    registry_median_base_fee: Decimal | None,
) -> list[RoutingCandidate]:
    """
    Called after cross-encoder reranking, before Semantic LLM selection.
    Mutates routing_score on each candidate in-place.
    """
    # Hard gates
    candidates = [
        c for c in candidates
        if c.success_rate >= SUCCESS_RATE_FLOOR
        and uptime_gate(c.uptime_score, UPTIME_FLOOR)
    ]

    for c in candidates:
        in_bootstrap = c.execution_count < BOOTSTRAP_EXECUTION_COUNT

        norm_latency = (
            (c.p95_latency_ms / registry_median_latency_ms)

```

```

        if (registry_median_latency_ms and not in_bootstrap) else 1.0
    )
    norm_base_fee = (
        (float(c.base_fee_usd) / float(registry_median_base_fee))
        if (registry_median_base_fee and not in_bootstrap) else 1.0
    )

    c.routing_score = routing_score(
        success_rate = c.success_rate if not in_bootstrap else
BOOTSTRAP_SUCCESS_RATE,
        uptime_score = c.uptime_score,
        p95_latency_ms = c.p95_latency_ms if not in_bootstrap else
BOOTSTRAP_LATENCY_MS,
        base_fee_usd = c.base_fee_usd,
        norm_latency = norm_latency,
        norm_base_fee = norm_base_fee,
    )

    return sorted(candidates, key=lambda c: c.routing_score, reverse=True)

```

Semantic LLM system prompt addition:

When selecting the best agent from the ranked candidates, honour the routing_score field. When semantic match quality is comparable between candidates, prefer the agent with the higher routing_score. The routing_score reflects historical success_rate, response latency, and cost efficiency – all measured by the platform.

13. Database Schema Changes

13.1 User Model — Additions

```

model User {
    // existing fields unchanged...

    // NEW:
    arrears_usd          Float      @default(0)
    arrears_flag         Boolean    @default(false)
    wallet_address       String?    // Privy custodial wallet
address
    privy_wallet_id      String?    // Privy internal wallet
ID
    withdrawal_address   String?    // DEV only: payout target

```

```

address
  role          UserRole @default(USER)
}

enum UserRole {
  USER
  DEV
}

```

13.2 New: AgentInvocation Table

```

model AgentInvocation {
  id            String    @id @default(cuid())
  session_id    String
  user_id       String
  agent_id      String
  call_id       String    @unique
  status        InvocationStatus
  latency_ms    Int?
  base_fee      Decimal?
  platform_tokens Int?
  error_type    String?
  created_at    DateTime @default(now())

  @@index([agent_id, created_at])
  @@index([session_id])
  @@map("agent_invocations")
}

enum InvocationStatus {
  SUCCESS
  ERROR
  TIMEOUT
}

```

13.3 New: Transaction Table

```

model Transaction {
  id            String    @id @default(cuid())
  session_id    String
  user_id       String
  agent_id      String
  call_id       String    @unique
  base_fee      Decimal
  platform_cut   Decimal
  developer_payout Decimal
  latency_ms    Int
  status        TxStatus  @default(PENDING)
}

```

```

    created_at      DateTime @default(now())
    settled_at      DateTime?

    @@index([status, created_at])
    @@index([agent_id])
    @@map("transactions")
}

enum TxStatus {
    PENDING
    SETTLED
    FAILED
}

```

13.4 Agent Model — Additions

```

model Agent {
    // existing fields unchanged...

    // NEW (Phase 1 metrics, maintained by metrics_refresh cron):
    success_rate      Float    @default(0.70)
    uptime_score      Float    @default(1.0)
    p95_latency_ms    Int      @default(5000)
    execution_count    Int      @default(0)
}

```

13.5 Payment Model — x402 Removal

```

model Payment {
    id          String @id @default(cuid())
    agent_id    String @unique
    enabled      Boolean @default(false)
    base_fee    String? // USD value as string e.g. "0.50". Null = free agent.

    // REMOVED (all x402-related fields):
    // type, chain_id, recipient_address, asset,
    // token_address, default_price, currency, facilitator_url
}

```

13.6 SessionTranscriptEntry — Addition

```

model SessionTranscriptEntry {
    // existing fields unchanged...

    invocation_cost    Decimal? // base_fee charged for this call. Null for

```

```
MCP / unsuccessful
}
```

13.7 New: RegistryStats Table (for routing bootstrap graduation)

```
model RegistryStats {
  id                      String    @id @default(cuid())
  task_category           String    @unique
  median_base_fee_usd     Decimal?
  median_latency_ms       Float?
  agent_count             Int       @default(0)
  computed_at             DateTime @default(now())

  @@map("registry_stats")
}
```

14. Registry Changelog — x402 Removal

Component	Change
emerge.yaml schema	payment: section now only contains enabled: bool and base_fee: string . All x402 fields removed.
Universal Agent Manifest	economy.payment_model : remove X402 . Replace with EMERGE_NATIVE .
MCP Adapter (harvesting)	Remove x402 metadata extraction from tools/list . No pricing extracted from MCP agents.
A2A Adapter (harvesting)	Remove AgentCard.capabilities.extensions x402 parsing. Extract only base_fee from payment section if present.
Payment Prisma model	Remove: PaymentType enum, chain_id, recipient_address, asset, token_address, default_price, currency, facilitator_url . Add: base_fee String?, enabled Boolean .
Registry API response	payment field now returns { enabled, base_fee } only.
EmergeSDK	Remove x402 decorator (@x402), Shim payment enforcement, Macaroon generation. base_fee is plain metadata only.

Updated emerge.yaml payment section

```
payment:
  enabled: true
```

```
base_fee: "0.50" # USD per invocation. Omit or set "0" for free agents.
```

15. Wallet Architecture (Privy)

15.1 Wallet Type — Embedded Wallets with Server-Side Access

MetaOrcha uses **Privy Embedded Wallets with server-side access enabled**. This is distinct from Privy's separate "Custodial Wallets" product (which requires a licensed custodian partner like Bridge and is unnecessary here).

How it works: Privy generates an EVM wallet per user and stores keys in their TEE (secure enclave). Your backend holds an `authorization_key` derived from the user's JWT. Every wallet operation — transfer, sign — requires your backend to authorise it. Users never see or manage keys directly.

Default chain: Base Sepolia (testnet). The Privy Transfer API natively supports Base, Ethereum, Arbitrum, and Polygon. Avalanche Fuji is **not** in the Transfer API's supported chain list — if Fuji is required, all transfers must use the low-level RPC method (`eth_sendTransaction`) instead of the clean transfer helper. **Recommendation: switch default testnet to Base Sepolia** to use the Transfer API without workarounds. Mainnet migration to Base mainnet is a future milestone.

15.2 Credentials Required from Privy Dashboard

Create an app at <https://dashboard.privy.io>. You will need the following and must store them as env vars in Gateway:

Credential	Env Var	Description
App ID	<code>PRIVY_APP_ID</code>	Public identifier for your Privy app
App Secret	<code>PRIVY_APP_SECRET</code>	Used to authenticate server-side API calls. Never expose client-side.

Both are available under **Settings → API Keys** in the Privy Dashboard.

15.3 SDK Installation

```
# In gateway's pyproject.toml dependencies:
pip install privy-client

# Do NOT install privy-python-sdk — that is a deprecated third-party
# Indonesian e-signature SDK,
# completely unrelated to Privy wallet infrastructure.
```

15.4 Client Initialisation

```
# gateway/wallet/privy_client.py

from privy import PrivyAPI
import os

# Single shared instance – initialise once at app startup
_client: PrivyAPI | None = None

def get_privy_client() -> PrivyAPI:
    global _client
    if _client is None:
        _client = PrivyAPI(
            app_id=os.environ["PRIVY_APP_ID"],
            app_secret=os.environ["PRIVY_APP_SECRET"],
        )
    return _client
```

For wallet operations that require acting on behalf of a specific user (e.g. user-initiated withdrawals), generate a user signer from their JWT and update the authorization key before the call:

```
def get_user_signer(user_jwt: str) -> None:
    """
    Call this before any wallet operation that should be authorised
    by the user (i.e. user-initiated withdrawals).
    For platform-initiated operations (settlement payouts), omit this –
    the platform's app_secret is the authorization key.
    """
    client = get_privy_client()
    signer = client.wallets.generate_user_signer(user_jwt=user_jwt)
    client.update_authorization_key(signer.decrypted_authorization_key)
```

15.5 Wallet Operations — Full Reference

Create wallet on user signup

```
# gateway/wallet/privy_client.py

async def create_wallet(user_id: str) -> dict:
    """
    Called by Gateway during user registration.
    Creates an EVM embedded wallet on Base Sepolia.
    Returns: { id: str, address: str, chain_type: str }
    """
    client = get_privy_client()
    wallet = client.wallets.create(chain_type="ethereum")
```

```
# wallet.id      → privy_wallet_id (store in User table)
# wallet.address → wallet_address (store in User table, shown in UI)
return {"privy_wallet_id": wallet.id, "wallet_address":
wallet.address}
```

Get on-chain USDC balance

```
async def get_wallet_balance(privy_wallet_id: str) -> dict:
    """
    Fetches live on-chain balance via Privy accounts API.
    Used by GET /wallet/balance endpoint.
    Note: internal credits_usd in DB is the billing source of truth.
    This is for displaying on-chain wallet state in the UI.

    REST: GET https://api.privy.io/v1/accounts/{account_id}/balance
    Python SDK: use httpx directly as account balance is not yet
    exposed as a typed SDK method – call the REST API.
    """

    import httpx, os

    async with httpx.AsyncClient() as http:
        resp = await http.get(
            f"https://api.privy.io/v1/wallets/{privy_wallet_id}/balance",
            auth=(os.environ["PRIVY_APP_ID"],
os.environ["PRIVY_APP_SECRET"]),
            headers={"privy-app-id": os.environ["PRIVY_APP_ID"]},
        )
        resp.raise_for_status()
        return resp.json()
    # Returns: { total: {value, currency}, assets: [{symbol, amount,
price}] }
```

Transfer USDC (developer payout / user withdrawal)

```
from privy.lib.stablecoins.usdc import send_usdc

async def transfer_usdc(
    from_wallet_id: str,
    to_address: str,
    amount_usd: float,
    chain: str = "base_sepolia",
) -> dict:
    """
    Executes a USDC transfer from a platform-controlled wallet.
    Used by:
    - Daily settlement cron: platform wallet → developer wallet
    - Developer withdrawal: developer wallet → external address
```

```

Transfer is ASYNC – Privy processes it in background.
Returns a wallet action with status='pending'.
Poll GET /v1/wallets/{wallet_id}/actions/{action_id} for completion.

Supported chains for transfer API: base, ethereum, arbitrum, polygon
and their testnets: base_sepolia, ethereum_sepolia, arbitrum_sepolia,
polygon_amoy
"""

client = get_privy_client()

# Use the high-level transfer action API (handles gas, nonce,
broadcast)
import httpx, os
async with httpx.AsyncClient() as http:
    resp = await http.post(
        f"https://api.privy.io/v1/wallets/{from_wallet_id}/transfer",
        auth=(os.environ["PRIVY_APP_ID"],
os.environ["PRIVY_APP_SECRET"]),
        headers={
            "privy-app-id": os.environ["PRIVY_APP_ID"],
            "Content-Type": "application/json",
        },
        json={
            "source": {"asset": "usdc", "chain": chain},
            "destination": {"asset": "usdc", "chain": chain,
"address": to_address},
            "amount": str(amount_usd),
        },
    )
    resp.raise_for_status()
    return resp.json()
# Returns: { id: action_id, wallet_id, type: "transfer",
#           status: "pending", source, destination,
balance_changes, steps }

```

Poll transfer action status

```

async def get_transfer_status(wallet_id: str, action_id: str) -> str:
    """
    Poll after transfer_usdc() to confirm settlement.
    Returns: 'pending' | 'completed' | 'failed'
    Called by settlement cron after initiating batch transfers.
    """

    import httpx, os
    async with httpx.AsyncClient() as http:
        resp = await http.get(

f"https://api.privy.io/v1/wallets/{wallet_id}/actions/{action_id}",
        auth=(os.environ["PRIVY_APP_ID"],

```

```

os.environ["PRIVY_APP_SECRET"]),
    headers={"privy-app-id": os.environ["PRIVY_APP_ID"]},
)
resp.raise_for_status()
return resp.json()["status"]

```

15.6 Gateway Module Structure

```

services/gateway/
├── wallet/
│   ├── __init__.py
│   ├── privy_client.py      # SDK init + all Privy API wrappers (above)
│   └── routes.py           # FastAPI router: balance, fund info,
withdraw
└── webhook.py             # Privy deposit webhook handler

```

routes.py

```

# gateway/wallet/routes.py

from fastapi import APIRouter, Depends, HTTPException
from .privy_client import transfer_usdc, get_wallet_balance
from common.pricing.constants import PRICING_ENABLED

router = APIRouter(prefix="/wallet", tags=["wallet"])

@router.get("/balance")
async def get_balance(user=Depends(get_current_user), db=Depends(get_db)):
    """
    Returns internal credits_usd (billing source of truth) +
    on-chain USDC balance from Privy (informational).
    """
    on_chain = await get_wallet_balance(user.privy_wallet_id)
    return {
        "credits_usd": user.credits_usd,
        "arrears_usd": user.arrears_usd,
        "arrears_flag": user.arrears_flag,
        "wallet_address": user.wallet_address,
        "chain": "Base Sepolia",
        "on_chain_usdc": on_chain,
    }

@router.get("/fund")
async def get_fund_info(user=Depends(get_current_user)):
    """Returns wallet address and chain for manual USDC deposit (QR code
    in UI)."""

```

```
    return {
        "wallet_address": user.wallet_address,
        "chain":          "Base Sepolia",
        "asset":          "USDC",
        "note":           "Send USDC to this address. Balance credited on
confirmation.",
    }
```

```
@router.post("/withdraw")
async def withdraw(
    request: WithdrawRequest,
    user=Depends(get_current_user),
    db=Depends(get_db),
):
    """DEV only. Initiates USDC transfer from user's Privy wallet to
external address."""
```

```
# — Authorization Gateway (backend-enforced, never Privy-level)
```

```
if user.role != UserRole.DEV:
    raise HTTPException(403, "Users cannot withdraw funds")
if user.arrears_flag:
    raise HTTPException(400, "Clear outstanding balance before
withdrawing")
if user.credits_usd < request.amount:
    raise HTTPException(400, "Insufficient credits")
if not user.withdrawal_address:
    raise HTTPException(400, "No withdrawal address configured")
```

```
# — Execute via Privy Transfer API
```

```
action = await transfer_usdc(
    from_wallet_id=user.privy_wallet_id,
    to_address=user.withdrawal_address,
    amount_usd=float(request.amount),
)
```

```
# Deduct credits immediately; on-chain confirmation is async
user.credits_usd -= request.amount
await db.commit()
```

```
return {"action_id": action["id"], "status": action["status"]}
```

```
@router.get("/transactions")
async def get_transactions(
    page: int = 1,
    user=Depends(get_current_user),
```

```

        db=Depends(get_db),
    ):
        """Returns paginated Transaction records for the user's wallet UI."""
        txns = await db.execute(
            select(Transaction)
            .where(Transaction.user_id == user.id)
            .order_by(Transaction.created_at.desc())
            .limit(20).offset((page - 1) * 20)
        )
        return txns.scalars().all()

```

webhook.py

```

# gateway/wallet/webhook.py
# Privy fires this webhook on confirmed USDC deposits to any user wallet.
# Configure the webhook URL in Privy Dashboard → Webhooks.

from fastapi import APIRouter, Request, HTTPException
router = APIRouter()

@router.post("/webhooks/privy/deposit")
async def handle_deposit(request: Request, db=Depends(get_db)):
    """
    Privy sends a POST when a deposit is confirmed on-chain.
    Payload contains: wallet_id, asset, amount, tx_hash.
    We look up the user by wallet_id and credit their credits_usd.
    """
    payload = await request.json()

    # Verify the webhook is genuinely from Privy
    # (Privy signs webhooks – validate signature against
    PRIVY_WEBHOOK_SECRET)
    verify_privy_signature(request.headers, payload)

    wallet_id = payload["wallet_id"]
    amount = Decimal(str(payload["amount"]))

    user = await db.execute(
        select(User).where(User.privy_wallet_id == wallet_id)
    )
    user = user.scalar_one_or_none()
    if not user:
        raise HTTPException(404, "Wallet not found")

    user.credits_usd += amount
    await db.commit()

    return {"credited": str(amount), "user_id": user.id}

```

15.7 Settlement Cron — Privy Integration

```
# gateway/jobs/settlement.py

from gateway.wallet.privy_client import transfer_usdc, get_transfer_status

async def execute_developer_payouts(pending_txns: list[Transaction], db:
AsyncSession):
    """
    Called by daily settlement cron after acquiring Redis lock.
    Aggregates payout per developer, executes USDC transfers,
    polls for completion, marks transactions SETTLED.
    """
    # Aggregate by developer wallet
    payouts: dict[str, Decimal] = {}
    for tx in pending_txns:
        dev = await db.get(User, tx.agent.user_id)
        payouts.setdefault(dev.privy_wallet_id, Decimal("0"))
        payouts[dev.privy_wallet_id] += tx.developer_payout

    # Execute transfers (async – poll for completion)
    action_ids = {}
    for wallet_id, amount in payouts.items():
        dev = await get_user_by_wallet(wallet_id, db)
        action = await transfer_usdc(
            from_wallet_id=PLATFORM_WALLET_ID, # platform treasury wallet
            to_address=dev.wallet_address,
            amount_usd=float(amount),
        )
        action_ids[wallet_id] = action["id"]

    # Poll until all transfers complete (max 60s)
    await poll_until_complete(action_ids)

    # Mark transactions settled
    for tx in pending_txns:
        tx.status = TxStatus.SETTLED
        tx.settled_at = datetime.utcnow()
    await db.commit()
```

15.8 RBAC — Enforced at Backend, Not Privy

Privy has no concept of "User" vs "Developer" roles. All wallets look identical to Privy. Role enforcement lives exclusively in your backend Authorization Gateway.

Operation	User	Developer	Where Enforced
View balance	✓	✓	DB query filtered by <code>user_id</code>

Operation	User	Developer	Where Enforced
Fund wallet (deposit)	✓	✓	Open — any USDC deposit to address is credited
Withdraw to external	✗ 403	✓	role == DEV check in routes.py before Privy call
View transactions	✓ Own only	✓ Own only	DB query WHERE user_id = current_user.id
Receive settlement payout	N/A	✓ Automatic	Settlement cron, no user action needed

15.9 E2E Wallet Flow (Updated)

Step	Actor	Privy Call	DB Action
1	Gateway on signup	<code>client.wallets.create(chain_type="ethereum")</code>	Persist wallet, privy_wallet, wallet.address, wallet_address
2	User deposits USDC	None (user sends to address externally)	Privy webhook fires, user.credits.amount
3	SuperAgent per-turn	None (DB only)	PaymentSettlement deducts turn_cost, writes Transaction()
4	Gateway daily cron	<code>transfer_usdc(platform_wallet → dev_wallet)</code>	Transaction SETTLED, settlement set
5	Developer withdraw	<code>transfer_usdc(dev_wallet → external_address)</code>	user.credits.amount

15.10 Environment Variables (Gateway .env)

```
# Privy credentials (from dashboard.privy.io → Settings → API Keys)
PRIVY_APP_ID=your-app-id
PRIVY_APP_SECRET=your-app-secret

# Webhook verification secret (from dashboard.privy.io → Webhooks)
PRIVY_WEBHOOK_SECRET=your-webhook-secret

# Platform treasury wallet ID (the platform's own Privy wallet for settlement payouts)
PLATFORM_WALLET_ID=your-platform-privy-wallet-id
```

```
# Kill switch (disables all pricing logic across all services)
PRICING_ENABLED=true
```

SuperAgent does not require any Privy credentials. It only reads/writes `credits_usd`, `arrears_usd`, `arrears_flag` on the `User` record in the shared DB. All on-chain operations are exclusively Gateway's responsibility.

16. Wallet UI

A **Wallet** tab is added to the Settings page.

UI Element	Detail
Balance card	Shows <code>credits_usd</code> in USD. If <code>arrears_flag</code> = <code>true</code> : warning banner with amount owed.
Chain badge	"Base Sepolia Testnet". Shows <code>wallet_address</code> (truncated) with copy button.
Transaction list	Paginated list of <code>Transaction</code> records: agent name, <code>base_fee</code> , status, date. <code>GET /wallet/transactions</code> .
Fund button	Shows QR code + <code>wallet_address</code> for USDC deposit. Available to all roles.
Withdraw button	DEV only. Form: amount + withdrawal address. <code>POST /wallet/withdraw</code> . Hidden for <code>USER</code> role.

17. Phase 2 & Phase 3 — Forward Reference

Phase 2 — Implicit Feedback Loop (v1.5)

Phase 2 adds behavioural signal mining from Phase 1's `agent_invocations` data. No new user interaction required.

Signal	Detection	Interpretation
User retry (same task)	Embedding similarity > 0.85 between consecutive messages	Negative — agent likely failed
Session abandonment	No user response for > 30 min after assistant message	Weak negative
Satisfaction expression	Lightweight classifier on user's final message	Positive
Task completion rate	Checklist steps completed / total steps	Structural quality signal

Signal	Detection	Interpretation
Agent switching	User rephrases + PnD selects different agent for same intent	Negative for original agent

Routing upgrade: `success_rate` in the routing score is replaced by the F:C (Fulfillment:Cost) ratio:

F:C Ratio = Implicit Quality Score / base_fee
Routing_Score = (fc_ratio × uptime_score) / (norm_latency × norm_base_fee)

Personalised selection memory: PnD begins tracking per-user, per-task-type agent preferences.

Phase 3 — Trained SuperAgent (v2)

- Fine-tuned selection model: `f(task_embedding, agent_id, user_history, context) → success_probability`
- GNN workflow optimiser: learns which agent combinations and topologies lead to successful outcomes
- Transparent reasoning trace: exposes full selection rationale to users
- `outcome_bonus` primitive activated: verified fulfillment-contingent fee, platform takes 25% commission
- `emerge-sdk` deeper telemetry: internal tool chains, retry patterns, error recovery flows

18. Open Items

Item	Status	Notes
Commission rate	Proposed 20%	Needs validation against actual infrastructure cost per workflow
Stripe integration	Not started	Fiat → credits top-up. Potential money-transmission compliance review required.
Developer tier gates	TBD	Criteria for Public → Verified → Native tiers. Must be purely meritocratic (<code>success_rate</code> threshold, uptime %, min execution count).
Dispute resolution	Phase 2	At Phase 1: disputes update agent quality signals only. Refund policy TBD.
USDC gas fees (Base Sepolia)	TBD	Platform sponsors gas for developer payouts, or developers hold Sepolia ETH? Base Sepolia faucet available for testnet.
Fiat developer payout	TBD	Stripe Connect as fiat alternative to USDC payout. Based on developer demand.

